

Simulazioni n -body su GPU: approccio naive e Barnes-Hut

Relazione del progetto di GPGPU - Damiano Trovato

Abstract

Con la seguente relazione si documenta il processo di realizzazione di una simulazione n -body tramite soluzioni parallele su GPU. Si trattano quindi aspetti degni di nota, quali gli algoritmi, i metodi numerici, le ottimizzazioni e un'analisi delle performance. Le soluzioni presentate saranno due: un'approccio naive su GPU, e un approccio approssimato Barnes-Hut, sempre su GPU.

1 Dettagli generali

In questa prima parte della relazione, si trattano gli aspetti comuni a tutte le soluzioni trattate.

1.1 Aspetti matematici

Si offre una lista delle formule matematiche che rendono fattibili le simulazioni realizzate all'interno del progetto.

Legge di gravitazione universale La seguente formula detta l'attrazione che agisce tra due corpi rispettivamente di massa m_1 ed m_2 , situate a distanza R . Questa grandezza è direttamente proporzionale al prodotto delle masse e inversamente proporzionale al quadrato della distanza, tutto moltiplicato per una costante (di cui verrà modificato il valore ai fini della simulazione).

$$F = G \frac{m_1 m_2}{R^2} \quad (1)$$

Dalla forza all'accelerazione

$$F = ma \Rightarrow a = \frac{F}{m} \quad (2)$$

Dall'accelerazione alla velocità L'accelerazione è la derivata della velocità sul tempo, possiamo quindi ottenere

$$a = \frac{dv}{dt} \Rightarrow a dt = \frac{F}{m} dt = dv \quad (3)$$

fissando un valore di dt , tramite la variabile `delta_time`, andremo a integrare rispetto al tempo, ottenendo la velocità.

Dalla velocità allo spostamento

$$x(t) = x_0 + v dt \quad (4)$$

dove $dt = \text{delta_time}$.

1.2 Metodo numerico di integrazione: leapfrog

Il metodo numerico utilizzato nella simulazione, è noto come metodo **leapfrog**, o metodo della cavallina. Il nome deriva dal modo in cui si alterna l'integrazione della velocità a quella del tempo.

Questo metodo numerico, sfasando di $\Delta t/2$ l'integrazione della velocità e della posizione, ottiene, nel contesto delle simulazioni n -body, risultati più accurati a parità di operazioni. L'accuratezza guadagnata, rispetto a un banale metodo di Eulero, si manifesta nell'energia del sistema, che viene conservata (con oscillazioni trascurabili).

Pseudo-codice 1 Integrazione Leapfrog

```
1: Calcola la forza
2: Aggiorna la velocità per  $\Delta t/2$ 
3: for Numero di iterazioni do
4:   Aggiorna la posizione per  $\Delta t$ 
5:   Calcola la forza
6:   Aggiorna la velocità per  $\Delta t$ 
7: end for
```

2 Simulazione parallela naive su GPU

Il primo approccio alla simulazione di sistemi a n corpi che si è deciso di realizzare, è una soluzione parallela basata sull'algoritmo naive, con complessità computazionale $\mathcal{O}(n^2)$.

2.1 I kernel fondamentali

2.2 L'algoritmo complessivo

2.3 Valutazione delle performance

3 Simulazione approssimata Barnes-Hut su GPU

3.1 I kernel fondamentali

I primi kernel sono quelli dedicati alla definizione della dimensione della bounding-box (ossia della radice del quad-tree). Usiamo una riduzione parallela per trovare il minimo e il massimo dell'intero sistema particellare, di ciascuna delle due dimensioni. In questo modo, sarà possibile costruire un parallelepipedo che contiene l'intero sistema, per ciascun istante di tempo t .

Pseudo-codice 2 Kernel per la riduzione parallela (ricerca del minimo o massimo)

Require: array A , array B , dimensione dell'input n

```
1:  $j \leftarrow$  id globale del work-item
2:  $i \leftarrow 2j$ 
3: if  $i > n$  then return  $\triangleright$  Usciamo dal programma se il work-item è fuori dal dominio dell'input
4: end if
5:  $val \leftarrow A[i]$ 
6: if  $i + 1 < n$  then
7:    $val \leftarrow \min(val, A[i + 1])$   $\triangleright \max(val, A[i + 1])$  nella ricerca del massimo
8: end if
9:  $B[j] \leftarrow val$ 
```

Questo kernel sarà lanciato fino a quando il minimo (massimo) si troverà alla posizione 0 dell'ultimo array usato. Nel codice host, abbiamo deciso di usare due array, scambiando ogni volta l'array di input e quello d'output (una sorta di ping-pong tra gli array).

3.2 L'algoritmo complessivo

3.3 Valutazione delle performance